UVP · CÓMPUTO PARALELO DISTRIBUIDO

Carga dinámica

Balanceo de carga estático vs. dinámico
en una arquitectura Maestro–Trabajador

ALUMNOS

Jared Méndez Valencia | Daniel del Valle Andrade
Johan Lezama Núñez | Camila Serrano Trinidad

Ingeniería en Sistemas y Tecnología de la Información
Grupo 7C

Objetivos

01

Explicar

Cómo se reparte el trabajo para que un nodo lento no frene a uno rápido.

02

Comparar

El algoritmo estático vs. el dinámico, con paquetes según la velocidad.

03

Proponer

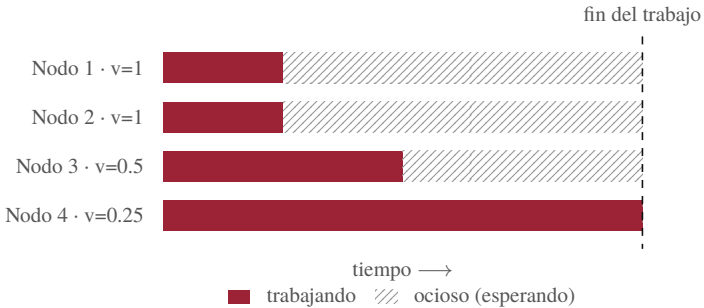
Estrategias para evitar nodos en espera.

04

Medir

Tiempo, speedup, eficiencia y ocio con Python y mpi4py.

Un nodo lento frena a uno rápido



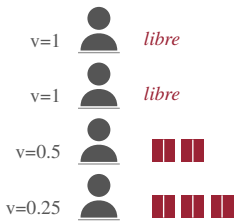
REPARTO IGUAL PARA TODOS

$$T = \max_i T_i$$

El trabajo termina cuando termina el más lento; los rápidos esperan.

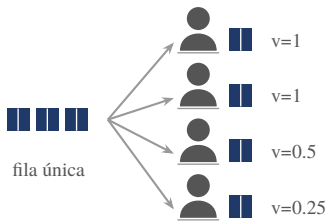
Una analogía: las cajas del súper

UNA FILA POR CAJA — estático



Mismo número de carritos para todas: las cajas rápidas se quedan sin trabajo.

UNA SOLA FILA — dinámico



La siguiente caja libre atiende al siguiente: nadie espera.

Conceptos y métricas

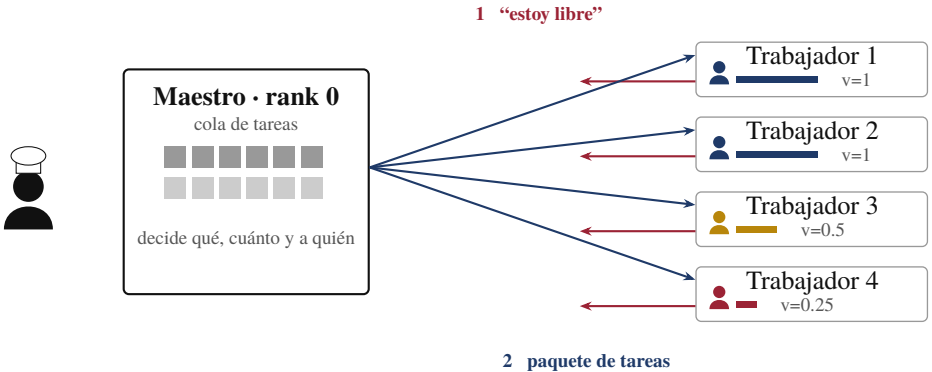
Nodo	máquina o proceso MPI
Tarea	unidad de trabajo con costo
Paquete	tareas entregadas juntas
Maestro	reparte y coordina
Trabajador	ejecuta y pide más
Heterogeneidad	velocidades v_i distintas

TIEMPO TOTAL	$T = \max_i T_i$
SPEEDUP	$S = T_{1\,\text{nodo}}/T$
EFICIENCIA	$E = T_{\text{ideal}}/T$
UTILIZACIÓN	$U = \text{ocupado}/T$

Balance ideal

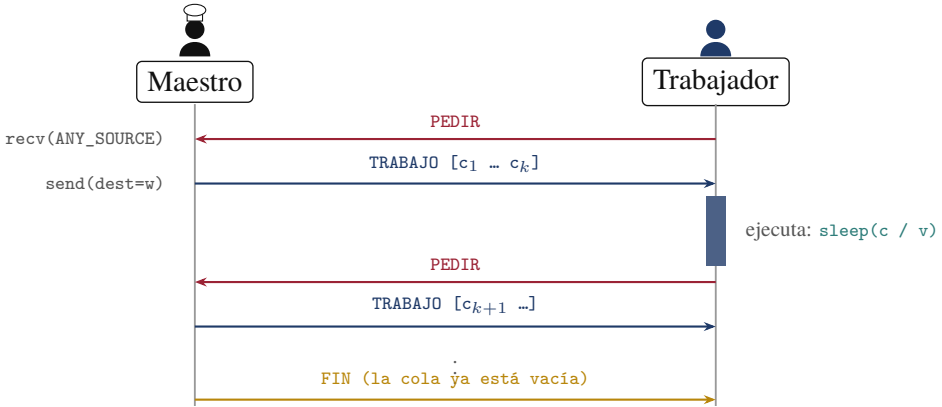
$T_{\text{ideal}} = \sum_j c_j / \sum_i v_i$ (todos terminan a la vez y no hay costo de comunicación)

Arquitectura Maestro–Trabajador



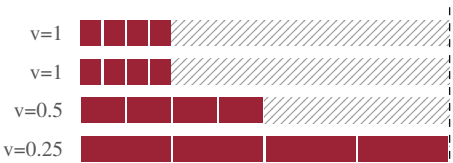
El maestro solo coordina; el que se desocupa pide más trabajo (esquema bajo demanda; Fernandes et al., 2024).

Protocolo de mensajes



Estático vs. dinámico

ESTÁTICO se decide antes de ejecutar



Las mismas 4 tareas para todos: el rápido espera.

1 mensaje por nodo (`Scatter`)

DINÁMICO pide el que se desocupa



Paquetes iguales: el rápido hace más y todos terminan juntos.

cola en el maestro · $\approx$ 2 mensajes por paquete

Algoritmos: estático y dinámico

ESTÁTICO

```
// MAESTRO
repartir las tareas entre los nodos
ENVIAR a cada nodo su lote

// NODO
lote ← RECIBIR
ejecutar todas sus tareas
BARRERA      // espera al más lento
```

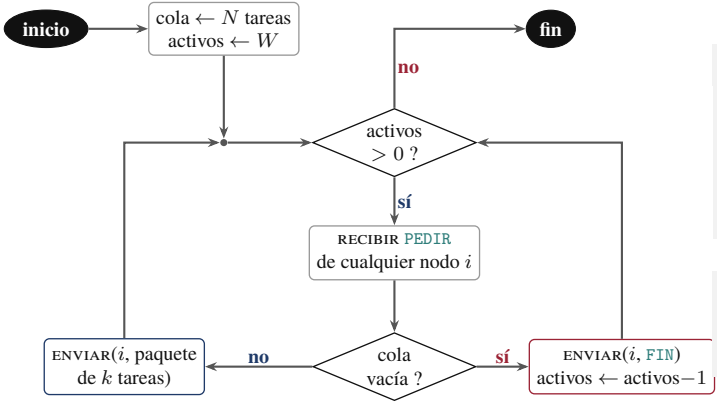
DINÁMICO

```
// MAESTRO
cola ← todas las tareas
MIENTRAS haya nodos activos:
    i ← RECIBIR(cualquiera)
    SI cola no vacía ENTONCES
        ENVIAR(i, paquete)
    SINO
        ENVIAR(i, FIN)

// NODO
REPETIR
    ENVIAR(PEDIR)
    paquete ← RECIBIR
    SI paquete = FIN ENTONCES salir
    ejecutar el paquete
```

En el estático el maestro decide todo al inicio; en el dinámico, **el nodo que termina pide más**.

Diagrama de flujo del maestro dinámico



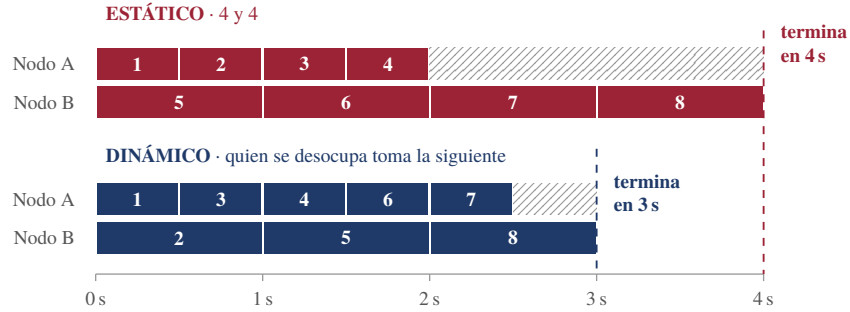
Invariantes

Cada tarea sale de la cola **una sola vez**. El ciclo termina cuando `activos` $= 0$.

Mensajes

$$M = 2\big(\lceil N/k \rceil + W\big)$$

Ejemplo paso a paso: 8 tareas, 2 nodos



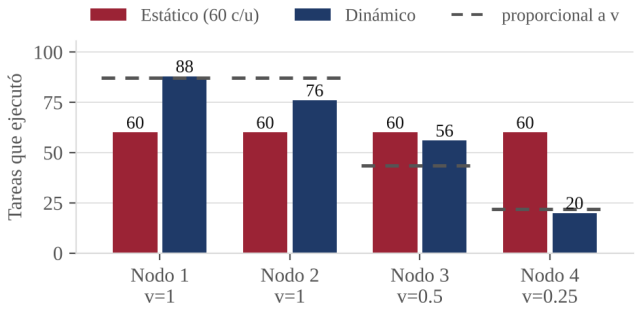
Estático 4 s → dinámico 3 s (25 % menos; el ideal sería 2.67 s)

Cuadro comparativo

CRITERIO	ESTÁTICO	DINÁMICO
Cuándo se reparte	Antes de ejecutar	Durante la ejecución
Mensajes	Pocos (1 por nodo)	Muchos ($\approx$ 2 por paquete)
Nodos distintos o tareas desiguales	Espera el más rápido	Se adapta solo
Un nodo que se degrada	No reacciona	Reparte menos a ese nodo
Implementación	Simple	Más compleja
Riesgo	Nodos en espera	Maestro como cuello de botella

Criterios: OpenMP ARB (2021) y Pacheco & Malensek (2022).

Paquetes según la velocidad



DINÁMICO

No se mide la velocidad: el nodo rápido **pide más veces**.

GUIADO

$$k = \max\left(k_{\min}, \left\lceil \frac{\text{restantes}}{f \cdot W} \right\rceil\right)$$

Paquetes **grandes al inicio**, chicos al final.

Equivale a las planificaciones dynamic y guided de OpenMP (OpenMP ARB, 2021).

Estrategias para evitar nodos en espera

1 Pedir bajo demanda

Nadie espera mientras haya cola.

En el proyecto

2 Paquetes guiados

Grandes al inicio, chicos al final.

En el proyecto

3 Ponderar por v_i

Si v_i es conocida y estable.

En el proyecto

4 Pesadas primero

Ordenar de mayor a menor costo (LPT).

Trabajo futuro

5 Prefetch

Pedir el siguiente paquete antes de terminar.

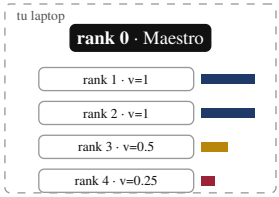
Trabajo futuro

6 Robo de trabajo

El ocioso roba tareas a otro, sin maestro (Fernandes et al., 2024).

Trabajo futuro

Simulación y diseño experimental



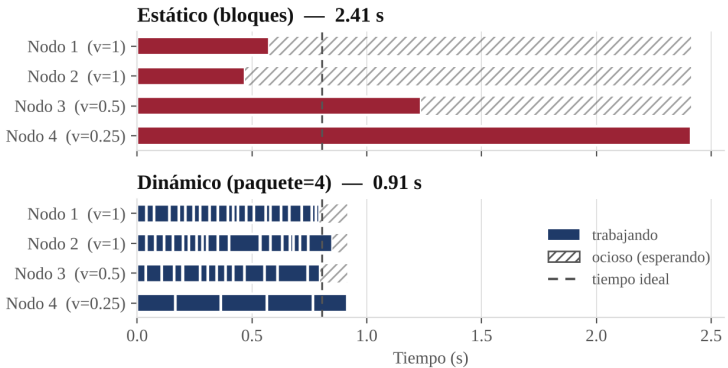
EN LA REALIDAD	EN LA SIMULACIÓN
Nodo	proceso MPI
Velocidad	factor v
Cómputo	`sleep(c/v)`
Red lenta	`--latencia-ms`
Nodo que se degrada	`--degradar`

ESCENARIOS MEDIDOS

1 nodos iguales 2 nodos distintos ($v = 1, 1, 0.5, 0.25$) 3 tamaño de paquete (1…60)
4 1, 2, 4 y 8 trabajadores 5 un nodo que se degrada

240 tareas · costo medio 10 ms (log-normal) · semilla fija · mediana de 3–5 corridas

Quién trabaja y quién espera

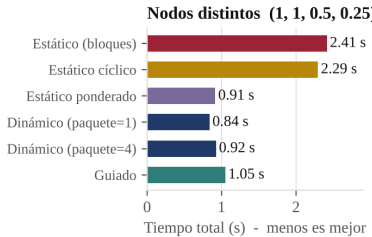


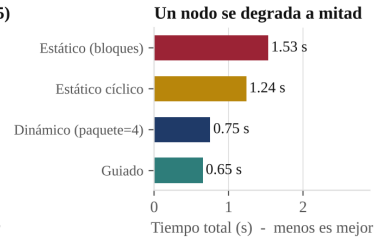
UTILIZACIÓN MEDIA

49 %
estático

91 %
dinámico

Tiempo total en dos escenarios



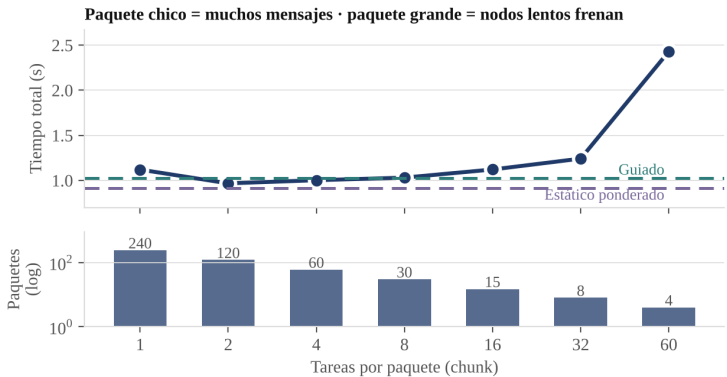


2.6×
más rápido con nodos distintos

2.0×
más rápido con un nodo degradado

0.63 s ≈ 0.62 s
con nodos iguales casi empatan

Tamaño del paquete



MEJOR PAQUETE

2

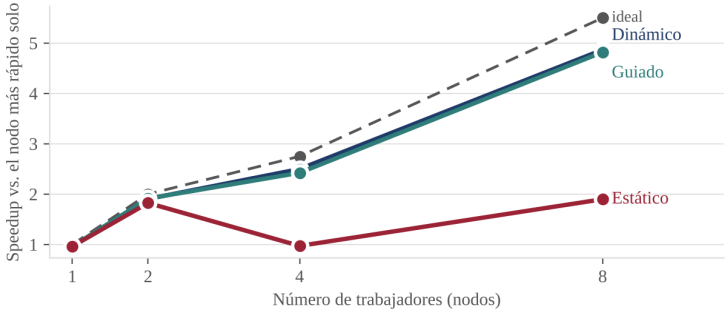
tareas → 0.97 s

PEOR PAQUETE

60

tareas → 2.43 s

Escalabilidad: más nodos

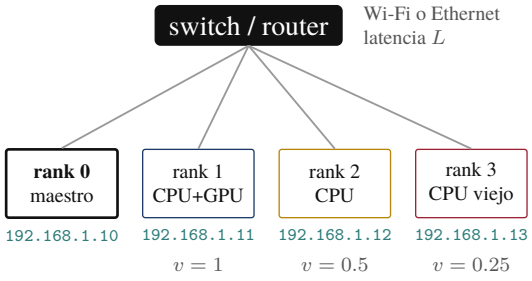


8 TRABAJADORES

4.9×
speedup dinámico

1.9×
speedup estático

Aplicación al clúster de laptops



Direcciones IP y velocidades v: ejemplo ilustrativo.

Ejecución en el clúster

```
# hosts.txt   (IP:procesos)
192.168.1.10:1   # maestro
192.168.1.11:1
192.168.1.12:1
192.168.1.13:1

mpiexec -f hosts.txt -n 4 \
  python balanceo.py \
  --estrategia dinamico --chunk 8 \
  --modo real --velocidades 1
```

Mismo código que la simulación: cambia solo `mpiexec`.

Latencia, paquete y balanceo CPU/GPU

UN PAQUETE EN LA RED



Sobrecosto de comunicación

$$f = \frac{2L}{2L + k\bar{c}/v} \leq 0.1 \;\Rightarrow\; k \geq 18\,L\,v/\bar{c}$$

Ejemplo: $L = 3\,\text{ms}$, $\bar{c} = 10\,\text{ms}$, $v = 1 \Rightarrow k \geq 5.4$. Con mensajes de pocos bytes domina la **latencia**, no el ancho de banda.

CPU, GPU Y VELOCIDAD RELATIVA

NODO	v_i	PAQUETES
CPU + GPU	1	más
CPU actual	0.5	medio
CPU viejo	0.25	menos

$v_i = \text{rend}_i/\text{rend}_{\text{máx}}$, medido con `cluster_diagnostico.py`

Qué balanceo usar

Dinámico: sin v_i · **ponderado:** con v_i

Conclusiones y trabajo futuro

1. **Estático:** simple y barato, pero un nodo lento frena a todos (2.41 s).
2. **Dinámico:** asigna paquetes a quien se desocupa, sin conocer velocidades (0.92 s).
3. **El paquete importa:** ni muy chico ni muy grande; el guiado no siempre gana (1.05 s).

Limitaciones

- Nodos simulados (`sleep`), una laptop
- Sin red física en los resultados
- El maestro puede ser cuello de botella

Trabajo futuro

- Prefetch con `isend/irecv`
- Robo de trabajo sin maestro
- Medir en un clúster real

¿Preguntas? · Referencias

¿Preguntas?

Gracias por su atención

Jared Méndez Valencia
Daniel del Valle Andrade
Johan Lezama Núñez
Camila Serrano Trinidad

Dalcin, L., & Fang, Y.-L. L. (2021). mpi4py: Status update after 12 years of development. *Computing in Science & Engineering, 23*(4), 47–54. https://doi.org/10.1109/MCSE.2021.3083216

Fernandes, J. B., de Assis, Í. A. S., Martins, I. M. S., Barros, T., & Xavier-de-Souza, S. (2024). *Adaptive asynchronous work-stealing for distributed load-balancing in heterogeneous systems* [Preprint]. arXiv. https://arxiv.org/abs/2401.04494

Message Passing Interface Forum. (2023). *MPI: A message-passing interface standard, version 4.1*. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report/mpi41-report.htm

OpenMP Architecture Review Board. (2021). *OpenMP application programming interface, version 5.2*. https://www.openmp.org/specifications/

Pacheco, P. S., & Malensek, M. (2022). *An introduction to parallel programming* (2nd ed.). Morgan Kaufmann.

Rogowski, M., Aseeri, S., Keyes, D., & Dalcin, L. (2023). mpi4py.futures: MPI-based asynchronous task execution for Python. *IEEE Transactions on Parallel and Distributed Systems, 34*(2), 611–622.